Project Documentation: ChatBot – Text & Voice Agents

1. Problem Statement

Many digital systems require users to navigate complex menus or interfaces to find information or complete tasks. This can be difficult for users who prefer natural conversation instead of manual interaction. The goal of this project is to develop an AI-powered chatbot that allows users to interact with systems using both text and voice. The chatbot understands user queries, provides intelligent responses, and enables more natural communication by replacing manual clicks with conversational commands.

2. System Architecture

The system follows a modern, decoupled client-server architecture designed for multimodal interaction (Text & Voice) and real-time database integration.

- **Frontend (User Interface):** Built with React, Vite, and TypeScript. It hosts the `ChatWidget`, capturing user input via keyboard, file uploads, or voice microphone. It listens for AI-triggered navigation events to seamlessly change UI views.
- **Backend (API & Logic Gateway):** Built with Python and FastAPI. It securely routes frontend requests, manages chat history for context, and handles business logic.
- **Database (Storage):** PostgreSQL (managed via SQLAlchemy). Stores application data such as Clients, Portfolios, and Meetings.
- **AI Engine (NLU & Decision Making):** The backend passes user queries and conversation history to the LLM. The LLM acts as an autonomous agent, utilizing Function Calling to execute database CRUD operations, perform Natural Language Understanding (NLU) data searches, or trigger frontend UI navigation.

3. How the System Works (Solution Workflow)

1. **Input:** The user interacts with the chatbot using text or voice.
2. **Speech-to-Text:** If voice is used, the browser's native Speech API converts the spoken audio into text.
3. **Context & Analysis:** The query, along with the previous conversation history, is sent to the AI system. The AI analyzes the intent (e.g., booking a meeting, querying high-risk portfolios, or navigating the dashboard).
4. **Action Execution:** The AI integrates with the application by calling backend functions (`analyze_financial_data`, `book_meeting`, etc.) to fetch or modify data.
5. **Output & Speech:** A response is generated using the language model. The response is returned to the UI as text, which is then converted to speech using **Text-to-Speech (TTS)** for voice output.

4. Key Features Implemented

- **Multimodal Input & Output:** Supports text-based typing, file/document context uploads, and voice-based conversations (Speech-to-Text / Text-to-Speech).

- **Context-Aware Conversation:** Maintains message history to understand follow-up questions without requiring the user to repeat previous information.
- **Natural Language Understanding (NLU):** Translates vague user intents (e.g., "Who are my riskiest clients?") into precise database queries.
- **Application Integration:** The chatbot actively controls the application. It can securely Create, Read, Update, and Delete database records, and even physically navigate the user's screen to different pages (e.g., "Take me to the dashboard").

5. AI Models & APIs Used

- **Language Model:** Google Gemini 2.5 Flash (`google.generativeai`). Chosen for its exceptionally fast response times and highly accurate Tool/Function Calling capabilities necessary for database manipulation.
- **Speech Processing APIs: Speech-to-Text (STT):** `window.SpeechRecognition` (Web Speech API) is used natively in the browser to transcribe user voice input into text in real-time.
    - **Text-to-Speech (TTS):** `window.speechSynthesis` (Web Speech API) is utilized to read the AI's generated text responses back to the user audibly.

6. Installation & Setup

Prerequisites: Python 3.9+, Node.js, and a PostgreSQL database.

Backend Setup:

1. Navigate to the backend directory: `cd backend`
2. Install dependencies: `pip install -r requirements.txt`
3. Create a `.env` file and add your credentials:

```
DATABASE_URL="postgresql://user:password@localhost/dbname"
GEMINI_API_KEY="your_google_api_key_here"
```

4. Start the server: `uvicorn main:app --reload` (Runs on `localhost:8000`)

Frontend Setup:

1. Navigate to the frontend directory: `cd frontend`
2. Install dependencies: `npm install`
3. Start the application: `npm run dev`

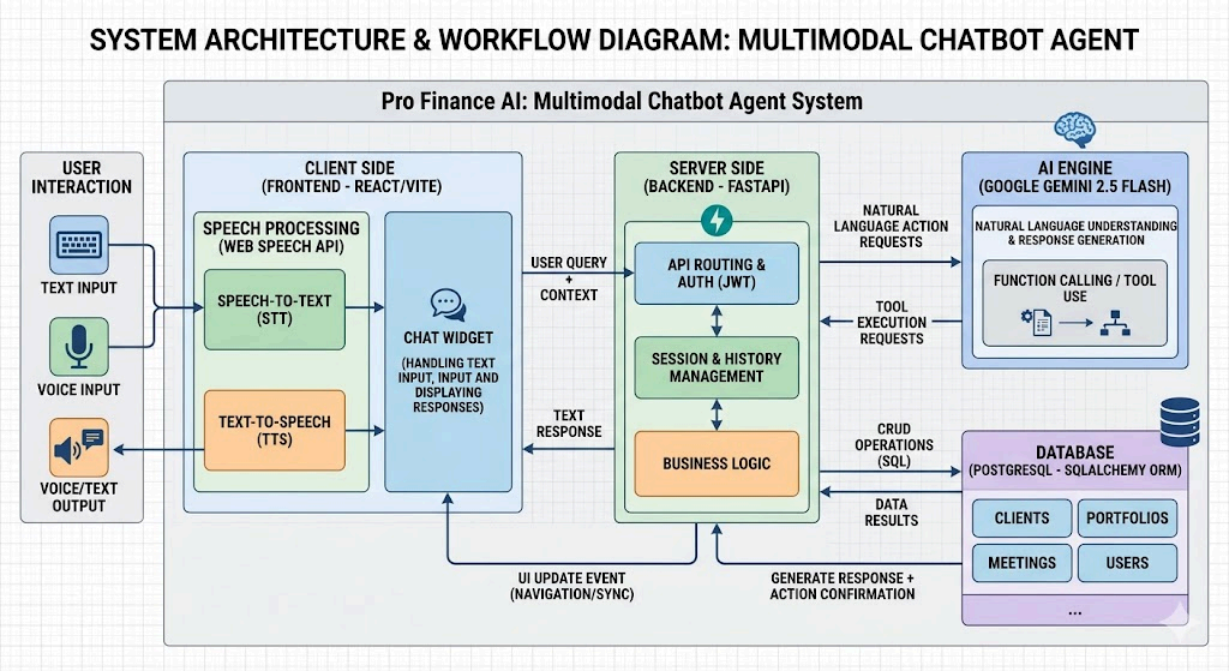
7. Example User Queries and Responses

Example 1: Natural Language Understanding (Data Query)

- **User (Voice/Text):** "Show me my high-risk clients."
- **System Action:** AI maps "high-risk" to the `analyze_financial_data` tool, querying portfolios with a risk score > 8.0.
- **Chatbot Response:**
  " **High Risk Portfolios:** - **sam :** Risk 9.0/10 | AUM: $700,000"

Example 2: Context-Aware Database Integration (Action)

- **User:** "Book a meeting with ram."
- **Chatbot Response:** "I can help with that. What date and time would you like to schedule the meeting, and which advisor will take it?" *(System recognizes missing constraints)*
- **User:** "Tomorrow at 2 PM with Sowmya." *(System uses history to remember 'John Doe')*
- **System Action:** AI calls `book_meeting` tool and saves it to the PostgreSQL database.
- **Chatbot Response:**
  "**Meeting Booked!** Scheduled for ram on 29 Mar 2026 at 14:00."

Example 3: System Navigation Control

- **User:** "Take me to the client directory."
- **System Action:** AI calls the `Maps_ui` tool with the parameter "clients". The frontend router triggers a page change.
- **Chatbot Response:** "Taking you to the Clients page!"